Kotlin Multiplatform — Practice Questions: Answers

Based on Lecture 8 (What Is Kotlin Multiplatform) and Lecture 9 (KMP Project Structure). Theoretical answers come from the slides; hands-on answers give the exact code/steps to do in Android Studio.

Section 1 — What Is Kotlin Multiplatform (Lecture 8)

Question 1

THEORETICAL

Task: Why KMP is neither a bridge nor a VM.

KMP is a **compiler technology**: shared Kotlin is compiled natively for each target, with no embedded runtime translating calls. The contrast:

- **Flutter** renders its own widget tree through **Skia/Impeller** — it doesn't use native platform views.
- **React Native** talks to the native views **across a JS bridge** (or the newer JSI).
- **Kotlin/Native** uses **LLVM to produce real ARM64 binaries** for iOS — not an interpreter. KMP skips the middle layer, so the UI can stay fully native (or be shared with Compose Multiplatform).

So to your teammate: KMP shares *logic* and compiles it to native code; it doesn't draw your UI or relay calls at runtime.

Question 2

THEORETICAL

Task: The “two codebases, one product” problem and its costs.

The Android team writes validation, networking and business rules in Kotlin; the iOS team writes the *same* validation, networking and business rules again in Swift. The three consequences listed:

- **Duplicated bugs**
- **Inconsistent behavior** between platforms
- **Double the QA effort** for every business-logic change

Question 3

THEORETICAL

Task: Shared percentage and shared-vs-native examples.

Typical shared amount: 60–80% of the codebase (the logic, not always the pixels).

Typically shared (commonMain)	Usually kept native — and why
Networking & API clients (e.g. Ktor)	Camera, biometrics, sensors — deep platform-hardware integration
Validation & business rules — logic that must behave identically	Push notifications — platform-specific delivery services

(Also shared: local storage/caching, state management. Also native: platform UI unless Compose Multiplatform is adopted, and some navigation for platform-idiomatic transitions and deep links.)

Question 4

THEORETICAL

Task: Stable vs Beta targets.

- **Stable:** Android; iOS (Arm64 + Simulator); Desktop (Windows, macOS, Linux); Server/backend (e.g. Ktor); Web — Kotlin/JS.
- **Beta:** Web — Kotlin/Wasm.

“Beta” means the target is usable but not yet fully mature — APIs/tooling may still change and rough edges remain, so it carries more risk for a production feature. Prefer the Stable targets for shipping; use Wasm for experiments or non-critical features.

Question 5

HANDS-ON — ANDROID STUDIO

Task: expect/actual getPlatform() on Android and iOS.

Create the project with the Kotlin Multiplatform wizard (or kmp.jetbrains.com) targeting Android + iOS, then use:

```
// commonMain/Platform.kt
interface Platform { val name: String }
expect fun getPlatform(): Platform

// androidMain/Platform.android.kt
class AndroidPlatform : Platform { override val name = "Android" }
actual fun getPlatform(): Platform = AndroidPlatform()

// iosMain/Platform.ios.kt
class IOSPlatform : Platform { override val name = "iOS" }
actual fun getPlatform(): Platform = IOSPlatform()
```

Build with **Build** → **Make Project** (or `./gradlew build`). Expected: no red underlines, and the `expect` gutter icon in commonMain jumps to both `actuals`. Open the three files side by side (drag a tab to split the editor) and take the screenshot — I can't run Android Studio, so that screenshot must come from you.

Question 6

THEORETICAL

Task: KMP vs Compose Multiplatform.

- **KMP shares logic** — networking, data, business rules, state. The UI can stay 100% native (Jetpack Compose on Android, SwiftUI on iOS).
- **Compose Multiplatform** is JetBrains' declarative UI framework that **additionally shares the UI layer**. It is **Stable on Android, iOS and desktop**; the **Wasm-based web target is Beta**.
- **Why one-way:** Compose Multiplatform is built on the multiplatform foundation (source sets, targets, expect/actual), so it needs KMP; KMP needs no UI framework. You can use KMP without Compose Multiplatform, but not the reverse.

Section 2 — KMP Project Structure Fundamentals (Lecture 9)

Question 7

THEORETICAL

Task: Why java.io.File fails in commonMain.

Common code is code shared across platforms, living in commonMain: one source file that the compiler turns into a set of platform-specific binaries — the same file compiled differently per target, not a copy.

`java.io.File` is part of the **JDK**. commonMain also compiles to native and JS targets where the JDK doesn't exist, so Kotlin refuses to resolve the reference. In a plain JVM project it works because JVM is the only target. Anything available only on some targets must go in a platform source set (e.g. jvmMain), where the same call compiles fine. The same applies to JVM-only extras of multiplatform libraries (e.g. kotlinx.coroutines' `asExecutor()`).

Question 8

HANDS-ON — ANDROID STUDIO

Task: Declare jvm() and iosArm64() targets.

```
// build.gradle.kts
kotlin {
    jvm()
    iosArm64()
}
```

Click **Sync Now**, switch the Project panel to **Project** view and open `src/`. Expected source set folders:

Folder	Meaning
commonMain / commonTest	Shared code and tests (all targets)
jvmMain / jvmTest	JVM target
iosArm64Main / iosArm64Test	iosArm64 target

(If the module also has Android or other targets declared, their folders appear too, e.g. androidMain.)

Question 9

THEORETICAL

Task: What makes a source set; root vs leaf.

A source set is a set of files with its own targets, dependencies and compiler options. The four properties:

- It has a **name that is unique** within the project.
- It contains **source files**, usually in a directory of the same name.
- It **specifies which targets** its code compiles to.
- It **defines its own dependencies and compiler options**.

commonMain is the predefined root source set present in every KMP project; it compiles to *all* declared targets, so it can only use APIs available everywhere. **androidMain** is a platform (leaf) source set that compiles to *one* target only, so it can use Android-only APIs — the same way jvmMain can use java.io.File.

Question 10

THEORETICAL

Task: The compilation merge rule and golden rules.

Merge rule: to compile for a target, Kotlin gathers **every source set labeled for that target** and compiles them together into one program. For iosArm64 that is commonMain + appleMain + iosArm64Main (for the JVM: commonMain + jvmMain).

Rules that follow:

- A target must be **declared** before Kotlin will compile anything to it.

- Code needed by **every** target belongs in commonMain; code for **one** target belongs in its own platform source set (and code for some targets in an intermediate set such as appleMain).
- Information flows **from common to platform**: platform source sets can see and use commonMain, but commonMain **cannot** see or reference platform source sets (this is why java.io.File fails there).
- Platform-specific behavior needed from common code is bridged with **expect (common) / actual (each platform)**, which the compiler merges into one API per platform.

Question 11

HANDS-ON — ANDROID STUDIO

Task: expect fun currentTimestamp().

```
// commonMain/Timestamp.kt
expect fun currentTimestamp(): Long

// androidMain/Timestamp.android.kt
actual fun currentTimestamp(): Long = System.currentTimeMillis()

// iosMain/Timestamp.ios.kt
import platform.Foundation.NSDate
import platform.Foundation.timeIntervalSince1970

actual fun currentTimestamp(): Long =
    (NSDate().timeIntervalSince1970 * 1000).toLong()
```

Note: `timeIntervalSince1970` returns seconds as a Double, so multiply by 1000 to match Android's milliseconds. Build with `./gradlew build` (or Make Project); both actuals resolve and there are no “expected declaration has no actual” errors.

Question 12

HANDS-ON — ANDROID STUDIO

Task: Add a js() target.

```
kotlin {
    jvm()
    iosArm64()
    js(IR) {          // on newer Kotlin versions: js { browser() }
        browser()
    }
}
```

Re-sync. Kotlin creates the matching source sets automatically — **jsMain** and **jsTest** now appear next to commonMain/jvmMain/iosArm64Main, with no manual wiring. Every common file now carries three target labels (JVM, iosArm64, JS) and compiles to a class file, a native executable and a .js file.

Question 13

HANDS-ON — ANDROID STUDIO

Task: Greeting class and GreetingTest.

```
// commonMain/Greeting.kt
class Greeting {
    fun greet(): String = "Hi"
}

// commonTest/GreetingTest.kt
import kotlin.test.Test
import kotlin.test.assertEquals

class GreetingTest {
```

```
    @Test
    fun testGreeting() = assertEquals("Hi", Greeting().greet())
}
```

Run via the green gutter icon next to the class/test, or `./gradlew allTests` (or `jvmTest`). Expected: the test passes. commonTest compiles to every declared target, so one test runs everywhere — matching the Main/Test pairing convention.